

EASY CODE

技术面试准备手册

从项目介绍，到架构追问，再到真实故障复盘

3 分钟

项目介绍口述稿

24 问

核心问题与示范回答

72 问

追问与继续回答

主线：让不确定的模型，在有权限边界、状态证据和预算约束的 Runtime 中工作。

本手册以当前代码和验证记录为依据。已实现机制、已覆盖测试与待验证的效果收益分别表述，不编造准确率、性能提升或个人履历。

先练主线，再练追问

建议：先看第 3-5 页，再优先练 Q06、Q08、Q15、Q17、Q24。

回答结构

结论一句话 → 当前实现 → 真实案例 → 边界与取舍。先回答 45-90 秒，追问时再展开源码；不要一开始就背类名和配置表。

01 为什么要做这个项目，而不是直接包装一个聊天 API？	6
02 一次用户请求从输入到完成，经过哪些步骤？	7
03 为什么强调“模型提出意图，Runtime 掌握权威”？	8
04 多 Provider 怎么扩展，又怎么避免到处写特殊分支？	9
05 为什么同时使用 Journal、Checkpoint 和 SQLite？	10
06 命令执行成功，但写完成事件前崩溃，Resume 怎么办？	11
07 项目有哪几种记忆？它们各自解决什么问题？	12
08 为什么不直接把 thinking 截短？现在怎么压缩？	13
09 摘要生成失败，为什么不会直接让长任务结束？	14
10 Token 预算怎么计算？多 Agent 同时请求会超支吗？	15
11 为什么要混合关键词和向量检索？	16
12 长期记忆如何避免错误积累和过期污染？	17
13 怎么让模型高效读代码，又避免错误修改？	18
14 命令权限为什么不能只做一张命令黑名单？	19
15 Benchmark 为什么 API 能联网，模型命令却不能联网？	20
16 超时后杀掉父进程，为什么仍然不够？	21
17 为什么 exitCode=0 不能直接代表测试通过？	22
18 怎么判断 Agent 在循环，而不是合理地继续调查？	23
19 模型修改测试后通过，系统怎么判断可信不可信？	24
20 Plan、DAG 和子 Agent 为什么不能混成一个概念？	25
21 Worktree 隔离解决了什么？结果怎么安全交回？	26
22 CLI 为什么会卡顿？你如何避免 UI 影响 Agent 执行？	27
23 你怎么测试这种带模型不确定性的系统？	28
24 讲一次最有代表性的故障，以及你的下一步改进。	29

第 30 页：模拟面试与复习路线 第 31 页：源码索引与不能说过头的结论

3 分钟项目介绍

按自然语速练习；若超过三分钟，先删实现名词，不要删问题、方案和边界。

0:00 - 0:30 | 项目定位

我做的是 EASY CODE，一个用 TypeScript 和 Node.js 实现的本地优先 CLI 编程 Agent。用户在项目目录里用自然语言描述目标，它可以调查代码、修改文件、执行测试，并在中断后恢复任务。项目的目标不是训练更强的模型，而是在现有模型之上，用可控的成本把长任务执行得更可靠。

0:30 - 1:10 | 核心架构

架构上，我把系统拆成 Provider 接入、可信 Runtime、工具执行、上下文记忆和任务编排几层。模型负责提出方案与结构化工具调用，Runtime 负责判断权限、校验参数、持久化状态和执行副作用。主任务和子任务共享请求及 Token 预算。关键事件写入追加式 Journal，Checkpoint 和检索索引用于加速恢复，但不能替代真实的执行记录。

1:10 - 2:10 | 两项技术重点

我重点解决了两个问题。第一个是长任务上下文膨胀：近期完整工具交互优先保留，较早交互整体摘要化，原始记录保留引用；摘要失败时按层级降级，而不是直接终止任务。第二个是有动作却没有进展：我把进程退出码和验证结论分开，通过原始工具结果、测试基线和失败签名识别重复失败，再触发受限的只读 reviewer。reviewer 必须提出可以执行的反证实验，不能只说换个思路。

2:10 - 2:45 | 工程验证

例如一次测试经过 grep 管道后，外层退出码是零，但测试摘要明确失败。此前这会被误判为进展；现在系统独立解析可归属的框架结果，不充分的证据保持 unknown。我也接入了 SWE-bench 的 Harbor 环境，把模型 API 通道和命令网络权限分开，命令不能联网查参考答案。

2:45 - 3:00 | 价值与边界

我认为项目的价值是把模型的不确定性约束在可审计、可恢复、有预算的执行流程里。当前机制已有自动化和容器级验证，但实际通过率、成本收益和误报率仍需要独立任务集对照，不能只凭机制设计就宣称优于其他 Agent。

30 秒备用版本

EASY CODE 是一个本地优先的 CLI 编程 Agent。我重点做的是可信 Runtime：统一模型工具调用、权限、预算、上下文降级与事件恢复；再用原始验证证据和只读 reviewer 减少长任务重复试错。模型负责提出动作，Runtime 负责决定能否执行、如何恢复和是否有足够证据。

用五层架构讲清项目

先说职责，再说依赖。模型、权限、持久化与 UI 不应互相越权。

层	负责什么	不能替代什么
CLI / TUI	接收输入、展示正文与 Thinking、菜单和状态	不直接决定权限或任务完成
可信 Runtime	模式、Schema、预算、调度、事件和恢复	不把模型声称成功当作事实
Provider / 模型 目录	协议转换、能力声明、响应与 usage	不为每家模型另建安全策略
工具 / 执行后端	读写校验、命令审批、隔离、监督和验证	Worktree 不能代替 OS 隔离
状态 / 记忆 / 检索	Journal、快照、长期记忆、历史召回	摘要和检索不能替代原始执行证据

一次请求的顺序

持久化输入 → 组装上下文与工具 → 预留并记录预算 → 请求模型 → 校验动作与权限 → 执行工具 → 记录结果和观察 → 继续 / 交付 / 可恢复暂停。

三种隔离，分别回答

Thread 隔离对话上下文；Worktree 隔离源码 Checkout；OS 后端限制进程能力。它们互补，不能把其中一个说成另外两个。

面试中的关键分界

事实层：原始执行事件、文件版本和允许的持久记录。建议层：模型方案、reviewer 假设、摘要与 RAG 命中。建议不能自行升级成权限或已验证结论。

这些数值可以说，但要说对单位

来自 src/config/runtime-defaults.json；用户配置、恢复状态和适配器可能覆盖默认值。

项目	当前默认	答题提醒
Actor 步数	none/low/medium: 40; high: 80	不是全系统总请求次数
共享模型请求	120	包含父、子及受预算约束的辅助请求
上下文字符	250000	不是 250000 Token
显式 Token 窗口	0	未配置额外 Token 上限
压缩触发 / 目标	80% / 55%	目标是软目标，还要检查实际容量
摘要提交 / 字段	最多 2 次 / 1200 字符	第二次超长可局部截断
摘要输出	2048 Token	不同于上下文窗口
近期交互优先保留	2 组	紧急降级可能减少
文件读取	默认 100 行；最多 1000 行	另受 12000 估算 Token 限制
查询 / 成功 / 失败输出	12000 / 2000 / 8000 字符	不是无限 stdout 保存承诺
子 Agent 并发	none/low: 2; medium: 4; high: 8	按当前强度选择，普通会话主动编排默认关闭
调查停滞窗口	12 响应、至少 5 样本、>70% 重复	先提醒，后续独立窗口才可审查

验证快照，不是效果宣传

本次 Harbor 修改的记录：主测试 1065/1065、扩展测试 28/28、适配器测试 6/6，另有真实 Docker 冒烟测试。尚未据此得到新的完整 benchmark 通过率，也不能推导具体 Token 节约百分比。

为什么要做这个项目，而不是直接包装一个聊天 API？

判断你是否理解 Agent 产品的工程问题，而不只是会调模型。

建议回答 | 先讲这一段

聊天 API 解决的是生成内容，编程 Agent 还要处理真实副作用：改错文件、命令失控、上下文膨胀，以及中断后不知道做到了哪里。我把重点放在 Runtime：统一工具契约、权限边界、预算和恢复，再让不同模型复用这些能力。项目的差异化目标是轻量、可控和可审计，而不是宣称基础模型能力更强。

追问 01 | 所谓轻量，具体轻在哪里？

主要是减少不必要的上下文和模型往返：工具输出分级投影、轮询在 Runtime 内等待、历史按需回读、默认关闭主动 DAG / 子 Agent 编排。轻量不等于依赖体积最小，本地 ONNX 模型和沙箱仍有安装成本。

追问 02 | 你怎么证明不是重复造轮子？

不以代码量证明价值，而以约束是否统一、故障是否可恢复、相同任务的成本和通过率来验证。已有库负责模型协议、检索和沙箱组件，我实现的是它们之间的执行契约与可靠性机制。

追问 03 | 最适合什么场景？

愿意使用自选 Provider、需要本地项目操作和可审计状态的开发者。它依赖远程推理，不是完全离线产品；是否适合敏感代码，还要考虑用户对 Provider 的数据使用约束。

不要说过头

不要把“本地优先”说成“代码从不离开本机”，模型请求仍会携带选中的上下文。

源码 / 文档核对

README_zh.md

docs/TECHNICAL_DESIGN_ZH.md §1-3

一次用户请求从输入到完成，经过哪些步骤？

能否讲清控制流、状态落盘和副作用顺序。

建议回答 | 先讲这一段

先校验配置、Prompt Bundle 和工作区并取得 Thread Lease，再持久化用户输入。Runtime 根据模式和状态组装上下文、裁剪可用工具、预留预算，调用 Provider。模型返回后先校验结构化动作，再做权限判断和执行，记录原始工具结果及派生观察，继续循环。完成不是模型输出一句完成就算数，还要满足当前 Plan、DAG 和结果提交约束。

追问 01 | Auto、Plan、Code 有什么区别？

Auto 是受限路由，选择直接回答、Plan 或 Code；Plan 用于只读调查和提交可审核方案；Code 执行实现与验证。模式不是提示词标签，工具集合和 Runtime 校验也随模式变化。

追问 02 | 用户在运行中追加要求怎么办？

追加调整先独立持久化，按 FIFO 在模型调用前后、工具之间等安全边界处理。不会任意打断正在提交的副作用，也不能通过追加文本扩大权限。

追问 03 | UI 可以直接改任务状态吗？

UI 应提交动作并展示投影，权威转换仍在 Runtime。否则终端刷新、重复按键或恢复界面都可能误改任务状态。

不要说过头

不要把 Auto 描述为简单关键词判断，也不要承诺每个最终回答都已通过独立评测。

源码 / 文档核对

src/runtime/agent.ts

src/app.ts

docs/TECHNICAL_DESIGN_ZH.md §3

为什么强调“模型提出意图，Runtime 掌握权威”？

信任边界、提示词注入与结构化工具设计。

建议回答 | 先讲这一段

模型输出本质上是不可信输入。文件、检索结果或命令日志可能包含诱导指令，所以不能让它们直接改变权限或任务完成条件。我的设计是在 Runtime 编译 Schema、角色能力、路径和状态校验；提示词只解释规则。即使模型声称用户已经批准，也必须查真实授权状态。

追问 01 | 有 Zod 校验是不是就安全了？

不是。Schema 只能验证结构，例如参数是不是字符串。还需要验证规范化路径、文件版本、权限、状态前置条件，以及 OS 层能否限制实际副作用。

追问 02 | 怎样对抗仓库 README 里的恶意指令？

把项目文本作为低信任数据，不让它覆盖系统权限、Provider 地址或敏感根目录。即使模型受骗发起操作，也要由工具边界和内核隔离阻断越界行为。不能宣称已经彻底解决所有提示词注入。

追问 03 | 为什么保留 Prompt Bundle？

它让规则与工具描述可版本化、可校验、前缀更稳定；但 Bundle 不是执行授权器，不能只靠修改提示词实现安全策略。

不要说过头

“提示词写了禁止”与“系统强制禁止”必须分开说。

源码 / 文档核对

src/prompt-bundle

src/tools/registry.ts

docs/TECHNICAL_DESIGN_ZH.md §1-2

多 Provider 怎么扩展， 又怎么避免到处写特殊分支？

接口抽象、协议差异和策略归属。

建议回答 | 先讲这一段

我把请求、响应、工具调用和 usage 统一成内部结构，协议差异尽量收敛在 Provider 边界和模型目录。上下文压缩、预算、命令权限、reviewer 都由共同的 Runtime 管理，而不是给每家模型写一套策略。目录描述模型能力与配置，不能把同一家供应商的不同计费渠道随意混用。

追问 01 | Provider 无关是不是意味着完全没有特殊处理？

不是。图片能力、thinking 字段和协议参数需要适配；我强调的是适配只负责转换，不负责另起一套安全和压缩算法。机制统一不等于协议完全相同。

追问 02 | 不同模型的 Token 计算也能统一吗？

使用共同预算接口和估算，并结合返回 usage 校准；它不是所有模型的精确 tokenizer。已知模型窗口应配置明确上限并留安全余量。

追问 03 | 重试放在哪一层？

由 Runtime 统一计入请求与 Token 预算，避免适配器隐藏重试造成超支。鉴权失败、限流、上下文超限与业务工具错误不能套用同一种重试。

不要说过头

不能把 GLM 普通 API 与 Coding Plan 说成同一凭据自动回退。

源码 / 文档核对

src/models/catalog.ts

src/runtime/task-budget.ts

docs/TECHNICAL_DESIGN_ZH.md §9

为什么同时使用 Journal、Checkpoint 和 SQLite?

事件溯源、派生状态和一致性边界。

建议回答 | 先讲这一段

三者解决的问题不同。Thread Journal 保存权威事件，Checkpoint 用来加速恢复，SQLite 提供查询投影、检索及长期记忆存储。恢复时不能拿旧快照覆盖更新的事件。也不能一概说 SQLite 都是缓存：长期记忆及修订的规范记录在 SQLite，向量和 Orama 索引才是可重建的派生数据。

追问 01 | 为什么不用一个 JSON 文件反复覆盖？

覆盖文件难以解释中断前发生了什么，也容易把一半更新误当成完整状态。追加事件保留转换顺序和审计线索，代价是需要版本校验、回放和快照管理。

追问 02 | JSONL 能保证数据库级的全部事务语义吗？

不能。它不是跨进程、跨文件的分布式事务。系统依靠单 Thread 所有权、明确提交顺序、事件校验和恢复策略降低不一致，而不声称所有外部副作用都能原子提交。

追问 03 | 日志损坏时怎么办？

不能凭模型摘要猜状态；应按现有完整性校验失败关闭并保留诊断。可重建的是派生投影，不是伪造缺失的事实。

不要说过头

区分 “Thread 事件事实来源” 和 “长期记忆数据库事实来源”。

源码 / 文档核对

src/storage

src/memory/vector-index.ts

docs/TECHNICAL_DESIGN_ZH.md §5-6

命令执行成功，但写完成事件前崩溃，Resume 怎么办？

副作用一致性与幂等性，这是高频深追问。

建议回答 | 先讲这一段

这属于结果不确定窗口。命令启动前先记录执行意图和租约，后续区分派发、目标退出与清理完成。如果缺少确定的完成证据，恢复不能静默重跑，因为上一轮可能已经修改文件或执行迁移。要保留现场、检查租约和工作区状态，必要时暂停。这里实现的是保守恢复，不是任意命令的 exactly-once。

追问 01 | 给命令一个 ID 就能实现幂等吗？

ID 只能帮助识别同一次操作和去重观察，不能自动让任意 shell 脚本幂等。真正幂等需要目标系统支持去重键，或者操作本身具备可比较的前后状态。

追问 02 | 哪些失败可以自动重试？

必须能证明尚未派发，并且清理已完成，再判断是否是可重试初始化错误。已经派发但结果未知，不能自动重试。

追问 03 | 用户删除 job 后为什么仍可能恢复旧状态？

job 展示记录与 checkpoint 存储不一定共用生命周期。当前适配器还绑定稳定 job 范围和任务身份。要检查实际匹配键和保留目录，而不是把删日志等同于清空所有恢复状态。

不要说过头

不要使用“有 Journal，所以副作用只会执行一次”这种绝对表述。

源码 / 文档核对

src/command/execution-journal.ts

src/command/runtime.ts

benchmarks/swebench_verified/easy_code_agent.py

项目有哪几种记忆？它们各自解决什么问题？

分清短期上下文、长期事实、检索与恢复。

建议回答 | 先讲这一段

我区分活跃上下文、工作摘要 / 恢复地图、长期记忆和历史检索。近期消息维持连续推理；摘要让早期交互退出活跃窗口；长期记忆保存跨 Thread 的偏好和有来源的项目事实；RAG 从私有历史中找相关片段。原始证据和 Runtime 状态另有权威存储，不能用记忆中的一句话代替执行结果。

追问 01 | 为什么不把所有历史都塞进向量库？

索引不等于应该检索全部内容。当前不索引系统指令和 thinking，自动历史检索主要针对已退出活跃上下文的部分，减少重复注入和不必要的噪声。

追问 02 | 压缩摘要会自动成为长期记忆吗？

不会。摘要要是连续性材料，不是经过长期记忆提交规则验证的事实。把二者混合会让未验证假设长期污染后续任务。

追问 03 | 记忆和 RAG 各有一份预算吗？

自动回忆共享一份有界预算，避免两边分别扩张。默认约 2000 个估算 Token，特定需要时可扩到 6000，并受项目容量和条目数限制。

不要说过头

RAG 是检索机制，不是另一种能够保证真实性的记忆。

源码 / 文档核对

src/context/memory-controller.ts

docs/TECHNICAL_DESIGN_ZH.md §6.1-6.4

为什么不直接把 thinking 截短？现在怎么压缩？

推理连续性、协议边界与有损恢复。

建议回答 | 先讲这一段

逐段截短近期 thinking 可能破坏尚在继续的推理和工具链，因此当前优先保留近期完整交互，选择较早且调用 / 结果配对完整的交互前缀做摘要。这个边界由消息结构决定，不要求测试已通过。容量不足时可以逐级移出历史，原始记录保留引用，并明确任务未完成、结论未验证。

追问 01 | 调查一直没完成，是不是永远无法压缩？

不是。已闭合的调查交互也可以退出上下文；闭合表示协议配对完整，不代表业务阶段结束。不能再把成功验证当成唯一压缩边界。

追问 02 | 默认保留两组近期交互是硬保证吗？

是优先策略，不是无限容量保证。极端压力下可减少保留范围，甚至做一次最小上下文重建；仍不能拆坏未闭合的工具协议。

追问 03 | 日志没删，是否代表压缩无损？

不代表。模型当前看不到全部历史，回读也依赖正确识别缺失信息。日志保留改善可追溯性，但不能保证推理质量完全不下降。

不要说过头

不能把“整体归档”说成“保存了完整推理效果”。

源码 / 文档核对

`src/context/exchange-boundary.ts`

`src/context/pressure-recovery.ts`

`src/context/compaction-transaction.ts`

摘要生成失败，为什么不会直接让长任务结束？

错误分级、有限重试和确定性降级。

建议回答 | 先讲这一段

我把可修复的长度超限与结构错误分开。每个压缩事务最多两次提交：第一次字段超长时给一次纠正机会，第二次仍超长就局部保留前 1200 字符，并标记有损。结构无效或容量仍不足则不继续付费循环，而是转入工具引用、历史退出、最小重建等确定性降级。不可压缩的必需内容仍放不下，才可恢复地暂停。

追问 01 | 为什么不直接忽略必填字段？

长度可以局部修复，但缺少当前工作或下一步会破坏摘要语义。不能通过猜字段、把字符串转布尔值等方式伪造有效候选，更不能认证无来源的结论。

追问 02 | 第二次只修改一个字段，其他字段会丢吗？

保留第一次候选，纠正可以是局部补丁，合并后再做完整校验。原始候选和具体长度诊断保存在 Journal，便于审计和恢复。

追问 03 | Resume 会重新获得两次机会吗？

不会。事务次数和已准备候选要随事件恢复；否则崩溃后重试就能绕过预算，重新进入反复摘要。

不要说过头

这里的局部截断针对摘要字段，不是把原始近期 thinking 任意截掉。

源码 / 文档核对

src/context/semantic-compaction.ts

src/context/compaction-transaction.ts

docs/PROGRESS_RELIABILITY_ZH.md

Token 预算怎么计算？

多 Agent 同时请求会超支吗？

资源预留、并发记账与成本口径。

建议回答 | 先讲这一段

我区分上下文容量、单次输出、Actor 步数和整任务共享预算。发送请求前，按估算输入加最大输出预留额度，并先持久化记账；同步预留防止多个子 Agent 同时花同一笔余额。返回 usage 后结算，缺失 usage 或崩溃中的请求保守消耗预留值。缓存和 reasoning Token 是 usage 子集，不能重复相加。

追问 01 | 默认 250000 是 Token 吗？

不是，是字符上限。maxContextTokens 默认为 0，表示未配置额外 Token 窗口，不是自动识别出模型有 25 万 Token。真正使用 Token 容量路径需要配置模型支持的上限。

追问 02 | 压缩到 55% 才能继续吗？

55% 是软目标，重点是有实际缩减且下一次正常请求能放下，包括工具定义和 Runtime 事实。还要预留下一轮输出与工具结果，避免刚压完又超限。

追问 03 | 更少输入 Token 一定更便宜吗？

不一定。如果丢信息导致更多调查轮次，总输入和输出可能反而增加。应比较相同任务集上的每成功任务 Token、耗时、通过率和失败成本，而不是只看一次请求。

不要说过头

本地预算是保守控制，不是供应商账单的精确复刻。

源码 / 文档核对

src/runtime/task-budget.ts

src/context/capacity.ts

src/config/runtime-defaults.json

为什么要混合关键词和向量检索？

检索收益、退化路径与证据可定位性。

建议回答 | 先讲这一段

代码场景有大量精确标识符，关键词检索适合函数名、异常名和路径；语义检索适合换一种说法描述的历史问题。因此使用 SQLite 词法检索与本地 Embedding 的语义候选，融合后去重、过滤和限额。每个分块保留来源、偏移和版本信息。向量不可用时退回词法检索，不让检索故障阻断主执行。

追问 01 | 本地向量模型是什么配置？

当前使用 multilingual MiniLM 的固定模型，输出 384 维向量。它服务于本地检索，不是聊天模型；Embedding 的分词窗口也不是聊天上下文的 Token 窗口。

追问 02 | 相似度高，为什么仍可能不该注入？

可能过期、与当前问题无关、已经在近期上下文里，或者只是表达相似。系统还要做文件版本、来源范围、重复覆盖和预算过滤，不能把向量分数当成事实置信度。

追问 03 | 怎么评估 RAG，而不是凭感觉调参数？

给固定查询建立相关证据标注，测召回与噪声，再观察真实任务的后续轮次、Token 和通过率。检索离线指标改善，不保证端到端任务效果必然改善。

不要说过头

不要把历史 RAG 说成外网搜索，也不要把向量缓存当成唯一持久记录。

源码 / 文档核对

src/memory/vector-index.ts

src/memory/memory-manager.ts

docs/TECHNICAL_DESIGN_ZH.md §6.4

长期记忆如何避免错误积累和过期污染？

记忆写入条件、来源约束和失效管理。

建议回答 | 先讲这一段

长期记忆不是模型随时写数据库。remember / revise 要提供允许的来源引用，先暂存，只有允许的回合完成结果才提交；失败或中断不会把提案固化为事实。项目事实自动注入前重新检查来源路径和文件哈希，过期就转为待验证。修改与遗忘必须使用本回合检索到的 ID，降低误操作。

追问 01 | 成功读过一个文件，就能保证记忆正确吗？

不能。来源校验只证明文件、版本和读取记录存在，不证明任意自然语言结论都能由其推出。因此记忆仍是来源的陈述，不应替代当前验证。

追问 02 | 为什么不是工具调用成功就立即写入？

一次 remember 成功只说明提案合规，任务可能随后失败或被用户纠正。延迟到回合允许的完成边界提交，减少半成品结论进入长期状态。

追问 03 | 向量写入失败要回滚事实吗？

有效记忆与修订在 SQLite 事务中提交，向量是派生数据，失败可稍后补建。检索性能退化与事实写入失败应分开处理。

不要说过头

不要说系统能自动判断所有记忆真假；也不要承诺用户删除 Thread 会自动删除所有工作区记忆。

源码 / 文档核对

src/tools/manage-memory.ts

src/memory/memory-manager.ts

docs/TECHNICAL_DESIGN_ZH.md §6.3

怎么让模型高效读代码，又避免错误修改？

工具可用性与读前写、路径、版本边界。

建议回答 | 先讲这一段

定位和理解分开：先用目录列表、文件名或文本搜索缩小范围，再读取相应片段。默认定位窗口 100 行，显式读取最多 1000 行，同时有 Token 上限。搜索结果只是位置，不是修改授权；修改仍要基于实际读取及版本校验。工具返回真实范围、文件哈希和继续读取位置，避免模型误以为看到了全文。

追问 01 | 为什么不把读取固定限制得很小？

太小会增加往返并破坏局部结构理解。合理方式是默认小窗口定位，确认位置后允许大段读取，同时限制总结果容量，而不是强迫每次只读几十行。

追问 02 | 一次简单项目介绍为什么会搜索很久？

递归搜索可能先陷入依赖、缓存目录并耗尽扫描上限。解决是提供单层 list 模式、广度优先搜索和明确的截断提示，不让模型把部分结果当成完整目录。

追问 03 | 字符串替换为什么容易出错？

JavaScript 的替换字符串会解释 `$&` 等特殊标记，可能把模型希望写入的字面量变成匹配内容。当前单次替换使用回调返回 `newText`；同时校验匹配次数，多匹配需要明确 `replaceAll` 或唯一上下文，不能猜要改哪处。

不要说过头

读取行数上限与 Token 上限同时生效，所以请求 1000 行不代表一定返回 1000 行。

源码 / 文档核对

`src/tools/read-file.ts`

`src/tools/search-files.ts`

`src/tools/update-file.ts`

`docs/LIGHTWEIGHT_RUNTIME.md`

命令权限为什么不能只做一张命令黑名单？

授权、语法校验和实际能力控制的区别。

建议回答 | 先讲这一段

同一段 Python 可以通过脚本、`python -c` 或 Shell 执行，黑名单很容易只限制写法而不限能力。因此先规范化 `argv`、`cwd` 和真实程序，保留多行等正常表达，再按风险审批，最后由隔离层限制能读写哪里、能否联网和子进程生命周期。审批解决用户是否同意，隔离解决实际能做什么，两者不能替代。

追问 01 | 三个审批模式怎么区分？

手动模式对可授权操作逐项审批；自动模式放行常规工作区操作和明确只读联网，高风险及下载 / 上传 / 未知联网仍审批；危险模式免审批，但仍受隔离、Plan 只读和 benchmark 禁网约束。

追问 02 | 允许前缀为什么要绑定可执行文件内容？

路径相同不代表程序未变。授权绑定真实程序、内容指纹和结构化参数，审批等待后再校验，避免程序被替换或把一个前缀误用于另一个动作。

追问 03 | 缺少 `verificationKind` 为什么不该拒绝执行？

它是验证分类元数据，不是确定程序所需的核心参数，可降级为 `custom` 或未知。缺少 `program` 则不能执行，因为不能猜用户和模型究竟要运行什么。

不要说过头

免审批不等于免安全边界；模型声明 `intent=inspect` 也不证明脚本只读。

源码 / 文档核对

`src/command/normalize-request.ts`

`src/command/runtime.ts`

`docs/COMMAND_SECURITY_ZH.md`

Benchmark 为什么 API 能联网，模型命令却不能联网？

控制面与执行面的网络隔离。

建议回答 | 先讲这一段

调用模型和执行模型生成的命令不是同一权限主体。Harbor 外层只允许固定模型服务端点，可信 Runtime 用它完成推理；命令子进程在专用后端里再受 seccomp 限制，不能创建 socket，所以连这个 API 端点也不能借用。安装依赖与最终可信 verifier 是独立阶段，不能把依赖下载当成模型查上游答案的通道。

追问 01 | Python urllib 或子进程 curl 能绕过吗？

单靠命令名识别挡不住，但 socket 限制由内核执行并被后代继承。启动前还关闭继承的描述符，避免通过父进程已有网络连接绕过。

追问 02 | 没有嵌套 namespace 怎么隔离？

Docker 作为外层边界，Harbor 后端使用 Landlock ABI 6+ 限制文件和信号能力，seccomp 限制网络与敏感系统调用。普通 CLI 保持原后端，不能因为容器标记存在就裸执行。

追问 03 | 这样的禁网会不会影响测试？

会。当前连本地 socket 也禁止，测试服务器或依赖 socket 的多进程功能可能失败。必须把这类基础设施 / 策略失败单独统计，不能偷偷放开外网来提高通过率。

不要说过头

不要说整个容器永远完全离线；可信安装、Runtime 模型调用和 verifier 有不同阶段权限。

源码 / 文档核对

src/sandbox/harbor-backend.ts

scripts/harbor-sandbox.c

benchmarks/swebench_verified/easy_code_agent.py

超时后杀掉父进程，为什么仍然不够？

后台进程监督、私有控制通道与清理确认。

建议回答 | 先讲这一段

子进程可能创建孙进程、脱离进程组或留下占用文件的后台任务。只看父进程退出，不能证明副作用停止。Windows 使用 Job Object；普通 Linux 使用沙箱 PID namespace；Harbor 的可信 supervisor 作为 subreaper 清理并回收后代。执行完成与清理完成分别记录；清理不确定就隔离后续操作，并禁止恢复 verifier 网络。

追问 01 | 为什么不用 stdout 打印一个 cleanup done?

stdout 受模型程序控制，能伪造，也会被裁剪。当前用不传给目标进程的独立控制管道，校验事件顺序，区分 ready、派发、退出和清理。

追问 02 | Harbor 的 setsid / 双重 fork 怎么处理？

subreaper 能接收被遗弃的后代。可信 supervisor 在主命令退出或收到取消信号后持续杀死并 wait 回收剩余子进程；超出清理期限则报告失败，不假装干净。

追问 03 | 取消、轮询和超时有什么不同？

轮询只是观察已有进程，不重复启动也不重置超时；取消是明确的终止动作；超时由原命令预算触发。最终结果要保留真实终态和清理证据。

不要说过头

真实烟测证明覆盖的场景通过，不等于证明所有内核和所有异常都绝无孤儿进程。

源码 / 文档核对

src/command/runtime.ts

src/command/windows-job.ts

scripts/harbor-sandbox.c

为什么 `exitCode=0` 不能直接代表测试通过？

项目最有说服力的具体问题与修复案例。

建议回答 | 先讲这一段

Shell 管道常返回最后一个过滤器的退出码。例如测试输出 `FAILED`，但 `grep` 找到了文本而返回零。之前将它记为 `passed`，会清除失败记录并误导 `ProgressGuard`。现在进程状态与验证状态分开，解析可归属的框架终态；明确失败可以覆盖外层零退出，输出缺失、目标混杂或解析不完整就保持 `unknown`。

追问 01 | 解析发生在 `grep` 之前吗？

不是。它发生在 `EASY CODE` 自己裁剪输出之前，但接收到的仍是 Shell 管道输出。`grep` 已经删除的证据无法恢复，必要时由模型选择不丢关键证据的最小验证。

追问 02 | 只看到 `FAILED(failures=2)` 就能触发 `reviewer`？

能认定这不是通过，但只有失败数量而缺少具体用例或断言，失败签名可能不够可靠。自动重复失败介入要求高置信证据，不能把所有失败摘要强行归为同一个问题。

追问 03 | 所有命令都必须解析测试框架吗？

不需要。直接 `build`、`lint`、`typecheck` 等可有退出码合约；不透明 Shell 或自定义验证则更保守。关键是不同证据来源对应不同置信度，而不是一刀切。

不要说过头

框架输出本身仍可被恶意脚本伪造，最终 `benchmark` 需要独立 `verifier`。

源码 / 文档核对

`src/command/verification.ts`

`src/progress/observation.ts`

`docs/PROGRESS_RELIABILITY_ZH.md`

怎么判断 Agent 在循环，而不是合理地继续调查？

可观测进展、阈值与误报控制。

建议回答 | 先讲这一段

我把已验证改善、新证据和完全重复分开。相同目标在三个不同验证周期出现相同高置信失败，是一个介入入口；重复调查则看同内容的读取覆盖、搜索结果和完整 inspect 输出。默认一个 12 次响应窗口至少 5 个成功样本且重复超过 70% 先提醒，后续独立窗口仍重复才请求 reviewer。时间长或没改文件本身不算停滞证据。

追问 01 | 换行号读同一文件，算新证据吗？

要结合内容哈希和区间覆盖。如果没有增加已知内容覆盖，只是换了行号窗口，不算新证据；真正的新范围可打断完全重复，但不等于已经改善方案。

追问 02 | reviewer 会不会和主模型一起胡思乱想？

它被限制为读取固定审查材料和限定源码，不能改文件或执行命令，输出必须包含证据、竞争解释和实验合约。结论只作为待验证假设，仍需主 Agent 执行对应实验。

追问 03 | reviewer 失败会反复花钱吗？

当前每个 scope 的自动审查次数受限，并共享已有请求 / Token 预算。Resume 不能重置 incident 或预算；也没有把自动提前终止全部任务作为默认机制。

不要说过头

不要称它为完整的“认知进展识别器”，仍有未覆盖工具和语义误判。

源码 / 文档核对

src/progress/guard.ts

src/progress/reviewer.ts

src/progress/experiment.ts

模型修改测试后通过，系统怎么判断可信不可信？

验证标准变化、基线绑定与作弊边界。

建议回答 | 先讲这一段

第一次执行修改、命令或子 Agent 管理工具前，固定当前 Thread 的测试和配置哈希基线，包含用户已有的未提交修改。之后比较验证前后的清单。修改原有测试、删除测试或改变收集配置，会使新通过不能直接解除旧失败；普通新增测试不自动判为作弊。清单不完整则无法确认标准未变。

追问 01 | 为什么不用 Git HEAD 当基线？

HEAD 可能不包含用户已存在的本地修改，回到 HEAD 会错误地把用户的合法验证标准排除。基线应该对应任务实际接手时的状态。

追问 02 | 这能识别所有削弱断言吗？

不能。它是常规测试路径和配置的有界哈希检测，不是语义分析。单条命令改完又恢复、自定义测试布局或伪造输出都可能逃过，因此仍需要独立评测。

追问 03 | 合法修复本来就需要改测试怎么办？

允许改，但不能用改变标准后的通过直接宣称原问题已恢复。应解释变更、保留原验证目标，并设计可比较的对照或独立验收。

不要说过头

测试变化是审查信号，不是直接给模型贴“作弊”标签。

源码 / 文档核对

src/progress/validation-standard.ts

src/progress/guard.ts

src/command/runtime.ts

Plan、DAG 和子 Agent 为什么不能混成一个概念？

规划审核、依赖调度和执行者职责。

建议回答 | 先讲这一段

Plan 表达实现方向并等待审核；DAG 表达执行依赖、检查和所有权；子 Agent 是负责某个有界任务的执行者。Runtime 校验图无环、依赖有效，一个节点不能有多个活跃所有者，完成要提交检查证据。子 Agent 使用私有 Thread 和受限能力，不能自己扩大权限或无限递归创建更多 Agent。

追问 01 | 为什么不是子 Agent 越多越好？

拆分会增加上下文复制、集成和协调成本。默认并发上限按父 Agent 当前 thinking 强度设置：none/low 为 2 个、medium 为 4 个、high 为 8 个，并共享总预算；适合边界明确、可独立验证的任务，不适合互相依赖的细碎步骤。

追问 02 | 关闭 orchestration 会把现有任务取消吗？

不会。它限制创建新的 DAG 和子 Agent，现有工作仍需管理和收尾；只读停滞 reviewer 是独立能力，不因关闭主动编排就一起关闭。

追问 03 | reviewer 为什么不直接用普通子 Agent？

普通子 Agent 有代码执行与交付职责，可能拥有写权限并参与节点所有权。reviewer 需要独立只读能力和不可变材料，不应抢占或完成被审查的 DAG 节点。

不要说过头

普通新会话默认关闭主动编排；benchmark 适配器明确开启，别把二者混为同一默认值。

源码 / 文档核对

src/subagents/coordinator.ts

src/tools/manage-tasks.ts

docs/LIGHTWEIGHT_RUNTIME.md

Worktree 隔离解决了什么？结果怎么安全交回？

上下文、源码和进程三种隔离的区别。

建议回答 | 先讲这一段

私有 Thread 隔离上下文，Git Worktree 隔离 Checkout，OS 沙箱隔离进程能力。Worktree 不是安全沙箱，也不自动同步父目录。子任务完成后形成包含基线、结果和依赖血缘的不可变 Artifact，Handoff 再检查冲突并显式交付。冲突时保留结果与环境，不覆盖用户未提交改动。

追问 01 | 非 Git 项目怎么办？

auto 可以使用共享工作区，并串行化写入、校验版本；显式要求 Worktree 而条件不满足时不能悄悄回退，因为用户选择的隔离语义变了。

追问 02 | 两个子任务都改同一个函数怎么办？

独立实现并不保证可合并。必须在集成时检测冲突，必要时由父 Agent 重新协调并验证；不能因为两边都声称成功就自动覆盖。

追问 03 | 重复 Handoff 会重复应用补丁吗？

要判断同一结果是否已经包含，并校验基线和血缘。可以安全重试的前提是识别同一结果，而不是任意重复执行 git apply。分支交付也不等于自动 push。

不要说过头

不要把 Worktree 描述成容器，也不要说它隔离了文件系统读取权限。

源码 / 文档核对

src/workspace/execution-environment.ts

src/subagents

docs/TECHNICAL_DESIGN_ZH.md §7

CLI 为什么会卡顿？ 你如何避免 UI 影响 Agent 执行？

终端渲染、输入所有权和性能诊断。

建议回答 | 先讲这一段

终端不是浏览器 DOM，反复重绘全部长文本和重复解析 ANSI 会占用事件循环，也可能打乱输入状态。当前把已完成内容写入 scrollback 一次，实时区局部重绘；thinking 展开是 UI 状态，不修改模型上下文。输入由单一组件持有，菜单退出要恢复草稿、光标和终端模式，非 TTY 则只追加输出。

追问 01 | 怎么定位是渲染慢还是模型慢？

分离 Provider 等待、工具耗时、渲染次数与耗时、事件循环延迟和输入响应。用固定长日志回放、滚动及 clear 场景复现，不用模型 Token 消耗速度推断终端性能。

追问 02 | 为什么 /clear 后可能连 Ctrl+C 都没反应？

可能涉及输入监听器、raw mode 或渲染循环没有正确释放，不能只归因于文本太长。回答要结合复现、事件所有权和实际修复证据，不凭现象断言唯一原因。

追问 03 | VS Code 点击展开 thinking 怎么通信？

通过经过认证的本地桥接传递 UI 动作，避免把控制文本直接塞进普通命令输入。展开状态不进入 RAG、长期记忆或下一次模型请求。

不要说过头

没有性能对照数据时，不要编造“提升十倍”或“零卡顿”。

源码 / 文档核对

src/cli

vscode-extension

docs/TECHNICAL_DESIGN_ZH.md §8

你怎么测试这种带模型不确定性的系统？

可确定性测试与端到端评测的分层。

建议回答 | 先讲这一段

把确定性机制和模型效果分开。Schema、预算、状态回放和策略用单元测试；Provider 通过可控响应模拟；文件、进程和隔离需要真实 OS / Docker 测试；最后再用固定 SWE-bench 子集做端到端评测。统计时分开环境安装失败、Provider 错误、任务验证失败和策略限制，不能把所有失败都解释成模型能力不足。

追问 01 | 1065 个测试通过说明什么？

这是本次修改后的主测试套件结果，另外有 28 个扩展测试、6 个适配器测试和真实 Docker 冒烟测试。它证明覆盖的工程行为通过，不等于 SWE-bench 通过率，也不证明模型成本一定降低。

追问 02 | 优化参数怎样避免对 benchmark 过拟合？

调参集和最终评估集分开，固定包哈希、模型配置、任务版本、并发、预算和网络策略。报告分母、失败类别与置信区间；预算允许时做重复实验，不能只挑成功样本。

追问 03 | 哪些指标比总 Token 更重要？

任务通过率、每成功任务 Token / 耗时、无进展调用占比、reviewer 实验执行率、错误提前退出率和基础设施失败率。缓存 Token、总输入和输出也要避免重复计数。

不要说过头

目前没有足够证据给出一条可对外宣称的最新准确率；面试前应重新整理实际结果。

源码 / 文档核对

tests

scripts/smoke-harbor-sandbox.mjs

benchmarks/swebench_verified/README.md

讲一次最有代表性的故障，以及你的下一步改进。

用事实解释因果、边界与个人贡献，避免只背术语。

建议回答 | 先讲这一段

我会讲 Harbor 安装失败：两个任务在 sandbox doctor 后 exit 78，agent_execution 和 verifier 都为空，所以问题发生在模型调用之前。追代码发现不只是环境变量设置得晚，Runtime 也固定选择内层沙箱。修复是把安装预检和执行统一接到专用 Harbor 后端，保留内核禁网与清理，再用真实容器验证。它是一次跨适配器和 Runtime 的契约修复。

追问 01 | 为什么不直接跳过 doctor 或开启特权 Docker?

跳过检查只能让安装继续，第一条命令仍可能失败；特权 Docker 又扩大宿主机风险。正确方向是实现可工作的明确后端并验证它，不是掩盖检查或削弱外层边界。

追问 02 | 这次改动有什么代价?

需要 Landlock ABI 6+、可信 helper 和额外安装编译步骤，且禁止本地 socket、限制部分元数据操作。已有烟测，但未证明所有 SWE-bench 环境和测试类型都兼容。要诚实暴露这些限制。

追问 03 | 下一步最值得做什么?

我会优先收集分层失败与成本数据，做不参与调参的长任务对照，再按证据改进停滞误报和命令兼容性。也要把 Runtime 中较大的控制流继续拆成可单测组件，而不是先增加更多 Agent。

不要说过头

使用第一人称前，确认哪些设计、实现和验证是你实际负责的；AI 辅助贡献如实说明。

源码 / 文档核对

benchmarks/swebench_verified/easy_code_agent.py

src/app.ts

src/sandbox/harbor-backend.ts

把手册变成能讲出来的答案

不需要背诵所有句子；要能解释每项设计针对的失败模式。

30 分钟模拟面试	练习内容
0-3 分钟	不看稿讲项目介绍，录音检查是否讲清问题与价值。
3-8 分钟	白板讲一次请求生命周期，解释三种隔离和谁拥有权威。
8-15 分钟	选择压缩、恢复或命令隔离，连续回答三层追问。
15-22 分钟	复盘 grep 误判或 Harbor 预检失败：现象、证据、根因、修复、验证。
22-27 分钟	讨论吞吐、成本、误报和安全之间的取舍；提出可测量的下一步。
27-30 分钟	说明个人贡献、AI / 开源工具使用方式，以及尚未验证的效果。

面试前 90 分钟复习

前 20 分钟：3 分钟介绍与架构。中间 40 分钟：Q06、Q08-10、Q15-19、Q24。再用 20 分钟打开对应源码，只核对自己会讲到的分支。最后 10 分钟检查版本、测试口径和个人贡献，不临时编造指标。

遇到暂时答不出来的问题

先澄清边界，再说明已知实现与未知部分。例如：‘当前实现对常规测试路径做哈希基线，但没有完整的断言语义分析。我会先构造反例，确认误判类型，再决定是否增加检查，而不是直接声称能识别所有作弊。’

个人贡献准备清单

列出你真实负责的需求判断、设计取舍、具体模块、故障定位和验证工作。对 AI 辅助生成的代码，准备说明你怎样审查、做边界测试和处理失败；不要把不能解释的实现包装成完全独立手写。

被追问“代码在哪”时，从这里找

相对路径均以项目根 F:/coding agent 为基准；源码比旧文档中的历史描述优先。

主题	优先阅读
请求循环与应用接入	src/runtime/agent.ts; src/app.ts
预算与默认配置	src/runtime/task-budget.ts; src/config/runtime-defaults.json
压缩与容量恢复	src/context/compaction-transaction.ts; semantic-compaction.ts; pressure-recovery.ts; capacity.ts
长期记忆与检索	src/context/memory-controller.ts; src/memory/memory-manager.ts; vector-index.ts
命令执行与验证	src/command/runtime.ts; verification.ts; normalize-request.ts
停滞与实验	src/progress/guard.ts; observation.ts; experiment.ts; validation-standard.ts
Harbor 接入	src/sandbox/harbor-backend.ts; scripts/harbor-sandbox.c; benchmarks/swebench_verified/easy_code_agent.py
协作与结果交付	src/subagents/coordinator.ts; src/workspace/execution-environment.ts
综合文档	docs/TECHNICAL_DESIGN_ZH.md; PROGRESS_RELIABILITY_ZH.md; COMMAND_SECURITY_ZH.md; LIGHTWEIGHT_RUNTIME.md

四句话必须守住

- 1. 有来源不等于已证明，模型建议不等于执行授权。
- 2. 日志可恢复不等于任意副作用 exactly-once。
- 3. 测试套件通过不等于 benchmark 准确率或成本收益。
- 4. 已实现、已验证和未来计划必须分别说明。

本手册为面试表达与复习材料，不替代项目安全审计。涉及精确性能、准确率、耗时或个人贡献的表述，请在面试前用自己的最新实验记录核实。